Homework 1: DDPM Implementation

CMU 10-799: Diffusion & Flow Matching

Spring 2026

Due: Saturday, January 24, 2026 at 11:59 PM ET **Total Points:** 80 + 20

Late Due Date: Monday, January 26, 2026 at 11:59 PM ET

Submission: Gradescope https://www.gradescope.com/courses/1207241

Starter Code: https://github.com/KellyYutongHe/cmu-10799-diffusion/

Note: *By popular demand, Q6 is now an extra credit question*

Introduction

Welcome to your first homework in CMU 10799 Diffusion & Flow Matching!

In this homework, you'll implement a Denoising Diffusion Probabilistic Model (DDPM) [Ho et al., 2020] from scratch and train it to generate realistic face images. But more than just "getting it to work," we want you to develop the intuition and debugging skills that researchers use every day. The structure of this homework mirrors a pseudo research workflow:

1. Understand your data: Before writing any model code, explore what you're trying to generate
2. Build intuition: Use the 1D playground to see diffusion in action (optional but recommended)
3. Implement: Write the core DDPM algorithm and U-Net architecture
4. Debug: Learn to diagnose common failure modes
5. Experiment: Run ablations to understand design choices
6. Reflect: Document what you learned and what you'd try next

This homework is AI-friendly. You may use any AI coding assistants, chatbots, or reference implementations. You may also use any other resources that you can find on the Internet. At the end of the homework, you'll document what resources you used.

Part 0: Setup your codebase (0 point)

First let's get started and setup your environments and codebase by forking and cloning the startup code repository: https://github.com/KellyYutongHe/cmu-10799-diffusion.

```
git clone https://github.com/KellyYutongHe/cmu-10799-diffusion.git
cd cmu-10799-diffusion

# Run setup script (auto-detects your hardware)
./setup-uv.sh     # Recommended (faster)

# Activate environment
source .venv-*/bin/activate
```

What You'll Implement:

File	What to implement
`src/data/celeba.py`	Data preprocessing transforms
`src/methods/ddpm.py`	Full DDPM algorithm
`src/models/unet.py`	U-Net architecture using provided blocks
`configs/`	Create your own model configs
`train.py`	Add sampling for logging
`sample.py`	Add your sampling scheme

What We Provide:

- **U-Net [Ronneberger et al., 2015] building blocks** (`blocks.py`): ResBlock, AttentionBlock, SinusoidalPositionEmbedding, Downsample, Upsample, etc
- **Training infrastructure**: Mixed precision, gradient clipping, checkpointing, logging
- **EMA**: Exponential moving average for stable sampling
- **Evaluation**: KID computation via torch-fidelity [Obukhov et al., 2020]
- **Modal integration**: Cloud GPU training for those without local GPUs
- **Notebooks**: Skeleton notebook for exploration
- **Scripts**: Example scripts to run your training and evaluation jobs on a slurm cluster or on Modal

Compute Budget:

You have **$500 in Modal credits** for the entire course (all 4 homework). Budget wisely!

Part I: Understanding Your Data (10 points)

Since the goal for generative modeling is to model the distribution of your data, it is important to first understand what does this distribution look like.

The dataset for this course is a custom subset of CelebA [Liu et al., 2015], filtered to have specific properties. Your first task is to discover what those properties are.

Q1. Visual Exploration [5 pts]

(a) [2 pts] Visualize a grid of at least 16 random samples from the training set. Include this grid below.

Figure 1: Grid of 16 random samples from the CelebA training subset.

(c) [3 pts] This dataset was filtered from full CelebA using their 40 binary attributes. Based on your visual exploration, hypothesize which attributes were likely used to create this subset. What filtering criteria were used to create this subset? Why?

By visually inspecting the training samples and cross-referencing with the CelebA `attributes.csv`, the subset appears to be filtered using the following criteria:

- **No blur:** All images are sharp and well-focused (`Blurry` = 0).
- **No obstructing accessories:** Subjects do not wear hats, goggles, make-up, or eyeglasses (`Wearing_Hat` = 0, `Heavy_Makeup` = 0, `Eyeglasses` = 0).

These filters ensure higher-quality training data for generative modeling by removing samples where facial features are occluded or image quality is degraded.

Q2. What did you learn? **[5 pts]**

(a) [3 pts] Imagine you found a pre-trained diffusion model that can reach SOTA performance on full CelebA (200K diverse faces). Now you generated samples and compared them to this filtered subset using KID or FID. Would you expect the FID and KID score to still be SOTA? Why?

No, the FID/KID scores would **not** remain SOTA. FID and KID measure the distance between the distribution of generated samples and the distribution of the reference set. A model trained on full CelebA learns to generate the full diversity of faces (with hats, glasses, makeup, blur, etc.), whereas the filtered subset occupies a narrower region of that distribution. The generated samples would therefore include modes not present in the reference subset, and the mismatch between these two distributions would result in degraded FID/KID scores—even though the model's generation quality is excellent.

(b) [2 pts] Based on your exploration, what kind of data augmentation transformation do you plan to use for your training and why?

The primary augmentation we plan to use is **random horizontal flipping**. Human faces are approximately left-right symmetric, so horizontal flips produce realistic samples that effectively double the dataset size without introducing artifacts.
Standard augmentations such as random cropping, color jitter, or Gaussian noise are **not suitable** here. Since a diffusion model learns to denoise images back to the data distribution, the training targets must be clean, high-quality images. Applying degradation-based augmentations would corrupt the ground truth that the model is trying to reconstruct, ultimately teaching it to generate lower-quality or distorted outputs.

Part II: Implement DDPM (25 points)

Now it's time to build your first diffusion model! Take a good look at the starter code and start building! Remember, you are free to add, delete and modify any and all parts of the starter code to fit your preference.

Q3. Building Intuition [optional, 0 pts]

Before diving into the full scale training, it is generally a good idea to build some intuitions of the algorithm with some toy examples as they are easier to debug.

(a) [0 pts] We have prepared a Jupyter notebook *01_1d_playground.ipynb* for you to experiment with 1D toy data. This notebook contains some options of mixture of Gaussians and their visualizations, you can try out your algorithm first in this playground.

(b) [0 pts] Full scale training can be difficult to debug sometimes, and that's why in *train.py* we also provide an argument flag, *--overfit-single-batch*, for you to toggle an experiment where you overfit to a single batch of data for sanity checking. You should be able to iterate your model a lot faster with this experiment than full scale training.

Q4. Implementation [25 pts]

Now implement DDPM for real! The starter code provides the skeleton, and your job is to fill in the core algorithm. Remember: this is your codebase. Feel free to modify any part of the starter

code to fit your preferences. Add helper functions, reorganize files, change the config structure... whatever helps. The only requirement is that your final code runs and produces good results.

The starter code provides a detailed guideline on which files are provided and which files are the ones that need your implementation. Go check it out!

(a) [5 pts] Train your DDPM model to convergence. You may use either the provided configs or your own. Report:

1. Model size
2. Batch size
3. Total training iterations
4. Training loss curve
5. Compute cost (GPU hours)

- **Model size:** 19,851,203 parameters
- **Batch size:** 256
- **Total training iterations:** 50,000
- **Training loss curve:** See Figure 2
- **Compute cost:** ∼20.7 GPU-hours (2× NVIDIA A4500, 10h20m each)

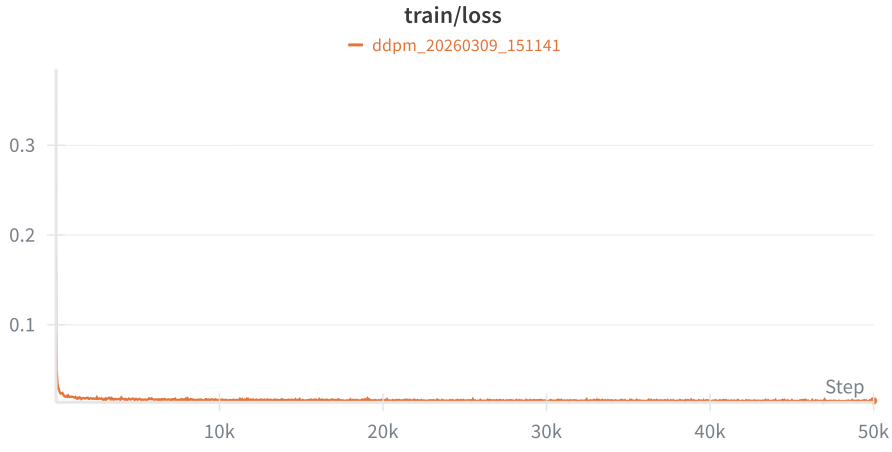


Figure 2: DDPM training loss curve over 50,000 iterations.

(b) [10 pts] Generate a grid of 16 samples from your trained model. Include this grid below.

Figure 3: Grid of 16 face samples generated from the trained DDPM model.

(c) [10 pts] Evaluate your model KID with 1k samples and report your KID score (both mean and std). You should be able to obtain KID < 0.005 with single L40S GPU training for a few hours.

KID score evaluated with 1k samples:

$$\text{KID} = 0.00333 \pm 0.00030$$

This is well below the target threshold of KID < 0.005.

(d) [0 pts] Provide a zip file of your code through Gradescope "Homework 1 Code" assignment. Make sure your code is runnable because we may run it to verify your results, and do not include any large files such as model checkpoints or dataset files. If you do not provide a zip file for your code, you will receive 0 point on Part II and Part IV of this homework.

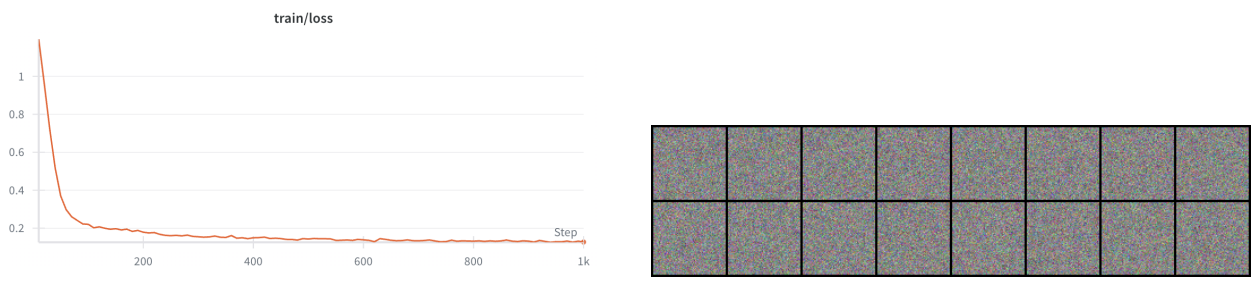
Part III: Help Your "Classmate" Debug (15 points)

Your classmate Kale is having some trouble getting her model to work. As someone who has successfully trained a DDPM model, you decide to help!

In each of the following scenario, you may be provided a loss curve, a grid of samples or a description of the situation. Try your best to help Kale with her implementation!

Q5. Pseudo Debugging Scenarios [15 pts]

(a) [5 pts] Kale trained a model for 1000 iterations and observed this seemingly nice looking loss curve. However, all her samples are pure noise. What could be the problem here? List at least two potential sources of bugs.



(a) The loss curve she observed (b) The final samples she obtained

Figure 4: The loss curve and the samples that Kale observed.

The loss curve is reasonable, so training is likely correct. The bug is in the **sampling** loop:

1. **Too few sampling steps.** Using far fewer timesteps than training (e.g. 10 instead of 1000) prevents full denoising.
2. x_T **not sampled from** $\mathcal{N}(0, I)$**.** Incorrect initialization (e.g. zeros or wrong scale) gives the reverse process an invalid starting point.
3. **Incorrect reverse process formula.** Wrong posterior mean/variance coefficients or missing the $\sigma_t z$ term would break denoising despite correct network predictions.

(b) [5 pts] Kale observed that after a while her loss would stay at a non-zero level, and it is quite "bumpy" – the loss does not decrease monotonically but instead has small fluctuations. Should she be worried about this? Why does this "bumpiness" exist? Can training for longer still be valuable for improving the model performance in her case?

She should **not** be worried. The bumpiness is inherent to the MSE loss: since it penalizes errors quadratically, a single poorly predicted sample can spike the batch loss, causing fluctuations even when the model is converging overall. A non-zero floor is also expected because perfectly predicting noise for every timestep and sample is impossible.
Training longer can still improve generation quality, as even small decreases in MSE correspond to better average denoising across timesteps, which translates to higher-fidelity samples.

(c) [5 pts] After the model fully converged, Kale obtained these samples shown below. What is her bug and how to fix it?

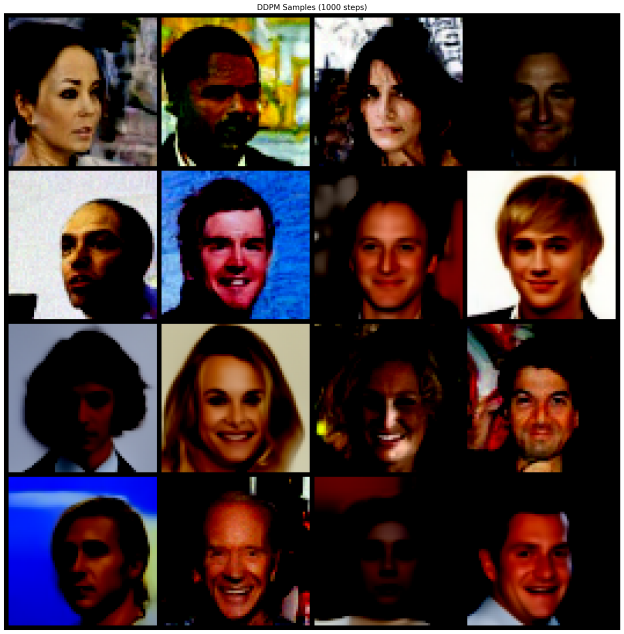


Figure 5: The samples that Kale obtained.

The images are scaled to $[0, 1]$ instead of $[-1, 1]$. DDPM assumes data normalized to $[-1, 1]$ so that the forward process endpoints match the standard normal prior. With $[0, 1]$ scaling, the noise schedule and reverse process coefficients are misaligned, producing washed-out or shifted samples. The fix is to rescale inputs to $[-1, 1]$ before training and reverse the scaling when displaying samples.

Part IV: Ablation Study (10 + 20 points)

In this part of the homework, we will explore different design choices of the DDPM algorithm and how they can affect the final performance.

Q6. Alternative Parametrization [optional extra credits, 20 pts]

Note: By popular demand, this question is now an option extra credit question. I.e. You don't have to do it if you don't want to. The homework now have a total of 80 points instead of 100. If you still complete that question you will receive 20 points extra credit.

(a) [7 pts] The original DDPM paper parametrized the model to predict the noise ϵ. However, this is not the only option! First derive an alternative parametrization for the same model (i.e. what else can the model predict that can still yield the same mathematical formulation of the algorithm).

In the standard DDPM ϵ-prediction parametrization, the posterior mean is:

$$\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\,\epsilon\right)$$

From the forward process $x_t = \sqrt{\bar{\alpha}_t}\,x_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon$, we can solve for ϵ:

$$\epsilon = \frac{x_t - \sqrt{\bar{\alpha}_t}\,x_0}{\sqrt{1-\bar{\alpha}_t}}$$

Substituting into $\tilde{\mu}_t$:

$$\begin{aligned}
\tilde{\mu}_t &= \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \cdot \frac{x_t - \sqrt{\bar{\alpha}_t}\,x_0}{\sqrt{1-\bar{\alpha}_t}}\right) \\
&= \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{(1-\alpha_t)(x_t - \sqrt{\bar{\alpha}_t}\,x_0)}{1-\bar{\alpha}_t}\right) \\
&= \frac{1}{\sqrt{\alpha_t}}\left(\frac{(1-\bar{\alpha}_t)\,x_t - (1-\alpha_t)\,x_t + (1-\alpha_t)\sqrt{\bar{\alpha}_t}\,x_0}{1-\bar{\alpha}_t}\right) \\
&= \frac{1}{\sqrt{\alpha_t}}\left(\frac{(\alpha_t - \bar{\alpha}_t)\,x_t + \beta_t\sqrt{\bar{\alpha}_t}\,x_0}{1-\bar{\alpha}_t}\right)
\end{aligned}$$

Using $\bar{\alpha}_t = \alpha_t \cdot \bar{\alpha}_{t-1}$, so $\bar{\alpha}_t - \alpha_t = \alpha_t(\bar{\alpha}_{t-1} - 1)$, and $\beta_t = 1 - \alpha_t$:

$$\tilde{\mu}_t = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\,x_t \;+\; \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\,x_0$$

Conclusion: The posterior mean depends on (x_t, x_0) directly, with no ϵ. Therefore, instead of predicting ϵ, the model can **predict** x_0. Given a network $\hat{x}_\theta(x_t, t) \approx x_0$:

$$\tilde{\mu}_\theta(x_t, t) = \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\,x_t \;+\; \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\,\hat{x}_\theta(x_t, t)$$

This is the x_0**-prediction** parametrization.

(b) [10 pts] Now implement your derived parametrization and train a model with the same configuration with your previous DDPM implementation with the only difference being the model parametrization. Include a grid of 16 samples and report your KID below. (Budget tip: You can compare the two parameterizations using shorter training runs, as long as they provide meaningful comparisons.)

Figure 6: Grid of 16 samples generated using the alternative (x_0-prediction) parametrization.

KID score evaluated with 1k samples:

$$\text{KID} = 0.01465 \pm 0.00050$$

(c) [3 pts] Compare the performance of your new parametrization with that of the original parametrization. Share your findings below.

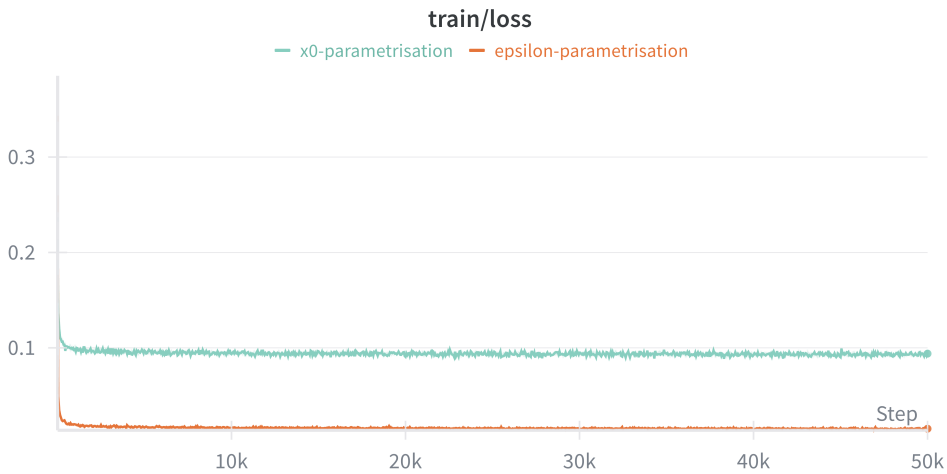


Figure 7: Training loss comparison: ϵ-prediction vs. x_0-prediction.

The ϵ-prediction parametrization clearly outperforms x_0-prediction, converging to a much lower training loss ($\sim$0.01 vs. $\sim$0.1). This is expected: in x_0-prediction the model must recover the clean image from a noisy input that entangles both signal and noise, which is inherently harder than predicting just the noise component ϵ. The KID scores reflect this gap (0.0033 for ϵ vs. 0.0147 for x_0).

Q7. Sampling Steps Ablation [10 pts]

(a) [10 pts] DDPM typically uses 1000 timesteps for sampling, which is slow. How about using fewer steps? Report the KID scores for DDPM sampling algorithm with $100, 300, 500, 700, 900$ steps and compare with your original sampling with 1000 steps. Include 1 sample for each number of steps to qualitatively show your comparisons as well.

Table 1: KID scores (mean $\pm$ std) for varying number of DDPM sampling steps.

Sampling Steps	KID Mean	KID Std
100	0.6289	0.00205
300	0.5859	0.00194
500	0.4648	0.00183
700	0.2101	0.00167
900	0.0282	0.00072
1000	**0.0033**	**0.00030**

Figure 8: Qualitative comparison of samples at different numbers of sampling steps.

Reducing the number of sampling steps significantly degrades KID. With 100 steps, KID is nearly 200× worse than the full 1000-step baseline. Quality improves sharply between 700 and 1000 steps, suggesting that the final denoising steps are critical for high-fidelity generation.

Part V: Reflection (10 points)

Now that you have had a fully immersive experience with DDPM, let's take a step back and reflect on what we have learned so far.

Q8. Reflection Questions **[10 pts]**

(a) [3 pts] Based on your experience implementing and experimenting with DDPM: What do you think DDPM is bad at? Identify 2-3 limitations you observed or suspect. These could relate to sample quality, training efficiency, sampling speed, controllability, or anything else. Be specific—connect each limitation to something concrete from your experiments or implementation.

The most obvious issue with DDPM is how **slow** it is—you have to run through hundreds or thousands of denoising steps just to get a single sample, which makes it way slower than something like a GAN that generates in one forward pass. On top of that, **scaling to higher resolutions** gets expensive fast since the model has to denoise at the full image size, so better quality basically means more compute. The other thing that stood out is the **lack of controllability**—there's no real way to tell the model what kind of image you want. It just samples from the learned distribution, so you can't say "generate a face with glasses" or anything like that without additional conditioning mechanisms.

(b) [3 pts] For those limitations you identified, how do you think they could be improved? Propose at least 2 concrete ideas. These don't need to be novel—they can be things you've heard about, ideas from papers, or your own speculation. For each idea, briefly explain why you think it might help. (We're not asking you to implement these—just to think critically about the design space.)

For the speed problem, one idea is to use a different sampling strategy like DDIM (Song et al., 2020), which reformulates the reverse process as a non-Markovian one so you can skip steps and get decent samples in 50 or even 10 steps instead of 1000. This works because

the denoising trajectory doesn't actually need to hit every timestep to produce good results. For controllability, you could train a class-conditional model by feeding the class label into the network (e.g., through adaptive group normalization like in Dhariwal & Nichol's guided diffusion work), so the model learns to generate specific categories. For even more open-ended control, text-guided approaches like DALL-E 2 and Imagen use CLIP or T5 text embeddings to condition the diffusion process, letting you describe what you want in natural language.

(c) [3 pts] What ablations or experiments are you still curious about? List 1-2 things you would explore if you had more time and compute budget. Why do these interest you?

One thing I'd want to explore is **alternative noise schedules**. During my experiments, the model seemed pretty sensitive to the schedule—I ended up sticking with the standard linear schedule from the original paper because other choices hurt performance. I'd be curious to try polynomial or cosine schedules (like the one proposed in Nichol & Dhariwal's improved DDPM paper) and see how they affect sample quality and training stability, especially at the endpoints where the linear schedule can waste steps on nearly pure noise or nearly clean images. The other thing I'd want to investigate is the **impact of the L_0 loss term**, which models the discrete log-likelihood of the final step predicting $\mathbf{x}_0$ from $\mathbf{x}_1$. I didn't implement it in my code, so my model skips that last step entirely. I'm curious whether adding it would noticeably improve the sharpness of generated samples, since that final step is where the model commits to actual pixel values.

(d) [1 pts] List all the resources that you found helpful for this homework. Include AI tools, open source code, tutorials, papers, classmates that helped you, etc.

AI tools: Tab-autocomplete of code snippets, debugging errors, especially for the visualization code in notebooks and getting the skip connection channel dimensions right in the UNet. GitHub Copilot for adding comments to my code.
Papers: The original DDPM paper (Ho et al., 2020) for architecture design and parameter choices. Improved DDPM (Nichol & Dhariwal, 2021) for ideas on what could be fixed/improved.
Blog posts: Calvin Luo's diffusion tutorial (https://calvinyluo.com/2022/08/26/diffusion-tutorial.html) and Lilian Weng's blog post (https://lilianweng.github.io/posts/2021-07-11-diffusion-models/) for understanding the math, verifying the alternate formulations of the variational bound.

That's it! You have completed HW 1! Congrats on your freshly baked diffusion models!

References

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.

Ziwei Liu, Ping Luo, Xiaogang Wang, and Xiaoou Tang. Deep learning face attributes in the wild. In *Proceedings of International Conference on Computer Vision (ICCV)*, December 2015.

Anton Obukhov, Maximilian Seitzer, Po-Wei Wu, Semen Zhydenko, Jonathan Kyl, and Elvis Yu-Jing Lin. High-fidelity performance metrics for generative models in pytorch, 2020. URL https://github.com/toshas/torch-fidelity. Version: 0.3.0, DOI: 10.5281/zenodo.4957738.

Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*, pages 234–241. Springer, 2015.